

Analýza toku dát

Ján Šturc, Richard Ostertág,
Jana Kostíčová

Kompilátory

O čom to je ?

- Počas kompilácie uvažujeme nad vlastnosťami a chovaním sa programu počas behu.
- Čo nás zaujíma
 - Vlastnosti, ktoré musia platiť vždy (invarianty) napr.:
 - V príkaze $x := y + z$ je y vždy rovné 1.
 - Pointer p vždy ukazuje do poľa a .
 - Príkaz S sa nikdy nevykoná.
 - Vlastnosti, ktoré platia často napr.:
 - Spravidla sa príkaz S_1 vykoná častejšie ako príkaz S_2 .
 - Vlastnosti, ktoré sa musia splniť aspoň raz počas vykonávania programu (intermitenty) napr.:
 - Počas vykonávania programu premenná x niekedy nadobudne hodnotu a .

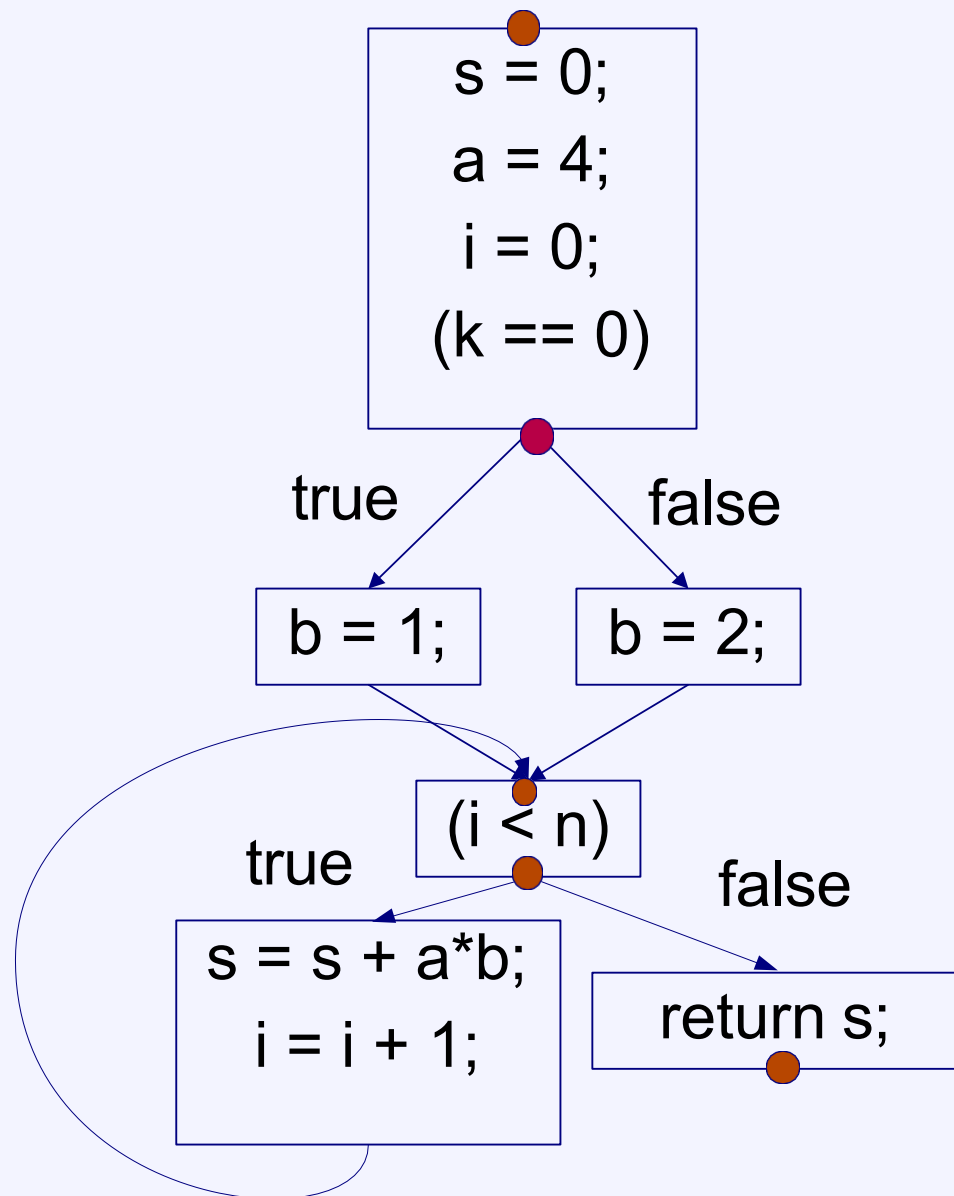
Graf toku riadenia (blokový diagram)

- Uzly reprezentujú základné bloky
 - Základný blok je postupnosť inštrukcií obsahujúca najviac jeden skok ako poslednú inštrukciu.
 - Všetky vstupy do základného bloku vedú cez jeho hlavičku (prvú inštrukciu)
 - Základný blok je (má byť) maximálny.
 - Dôsledok: Ak sa základný blok začne vykonávať vykoná sa celý
- Hrany reprezentujú vlastný tok riadenia (skoky a vetvenie)
 - Vzhľadom na definované inštrukcie je možné len binárne vetvenie.
 - Pri zovšeobecnených úlohách obmedzenie na vetvenie neplatí.

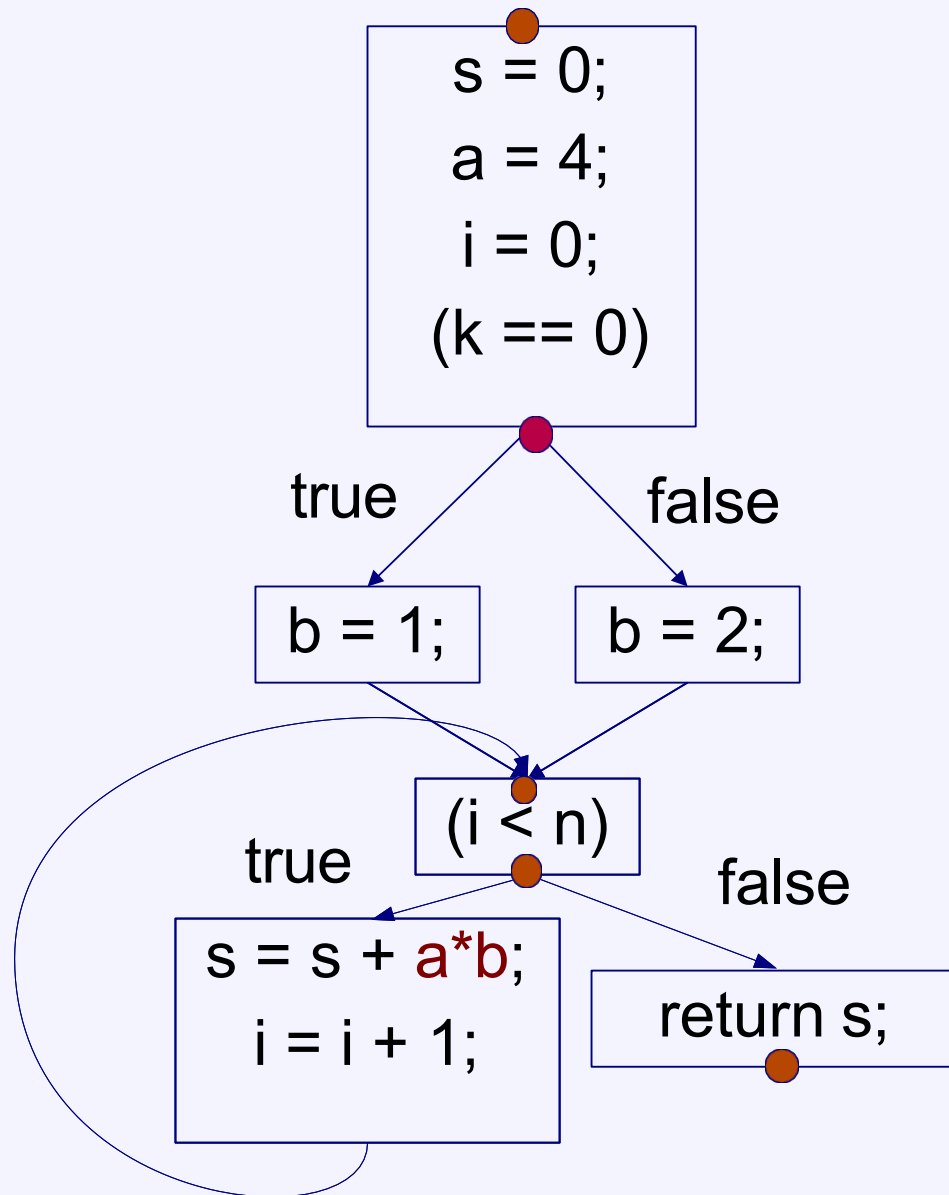
Príklad grafu toku riadenia

```
int add(int n, int k) {  
    s = 0; a = 4; i = 0;  
    if (k == 0) b = 1;  
    else b = 2;  
    while (i < n) {  
        s = s + a*b;  
        i = i + 1;  
    }  
    return s;  
}
```

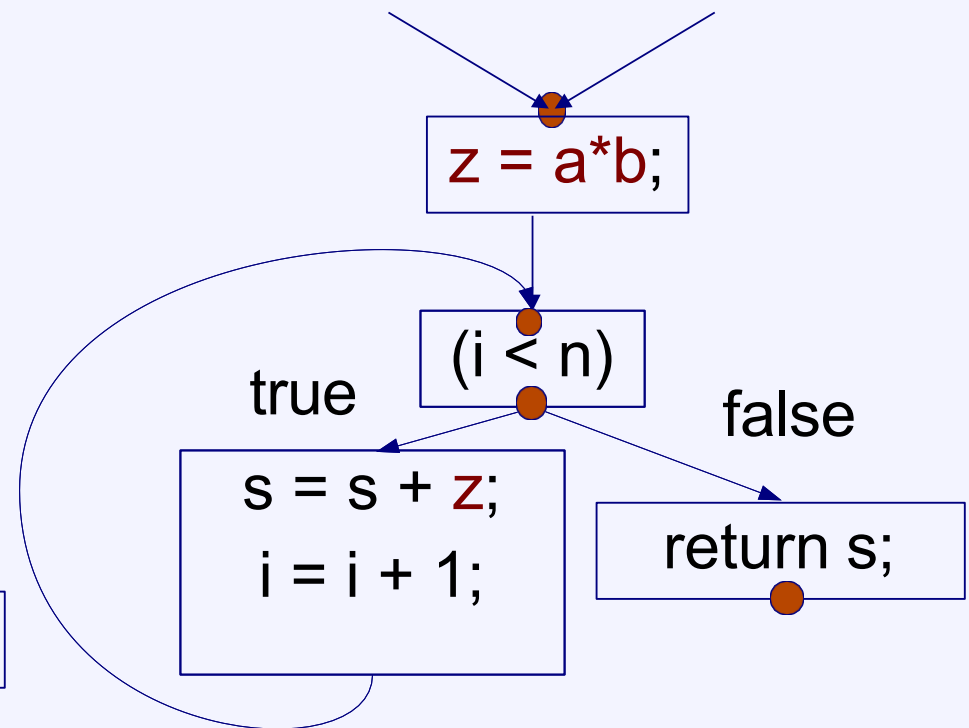
```
if (k == 0) return 4*n;  
else return 8*n;
```



Optimalizácia



$a*b$ nezávisí od premennej cyklu. Môžeme vypočítať raz, mimo cyklu.



Zaujímavé body grafu toku riadenia

- Vstup do programu n_0
- Body vetvenia (split)
 - $\text{succ}(n)$ = množina všetkých následníkov
 - Vzhľadom na definíciu inštrukcie je stupeň vetvenia vždy dva
- Body spájania (merge)
 - $\text{pred}(n)$ = množina všetkých predchodcov
 - Môžu byť ľubovoľného stupňa.
- Výstup z programu
 - n_F - jeden koncový uzol
 - Eventuálne môžeme mať aj množinu finálnych uzlov

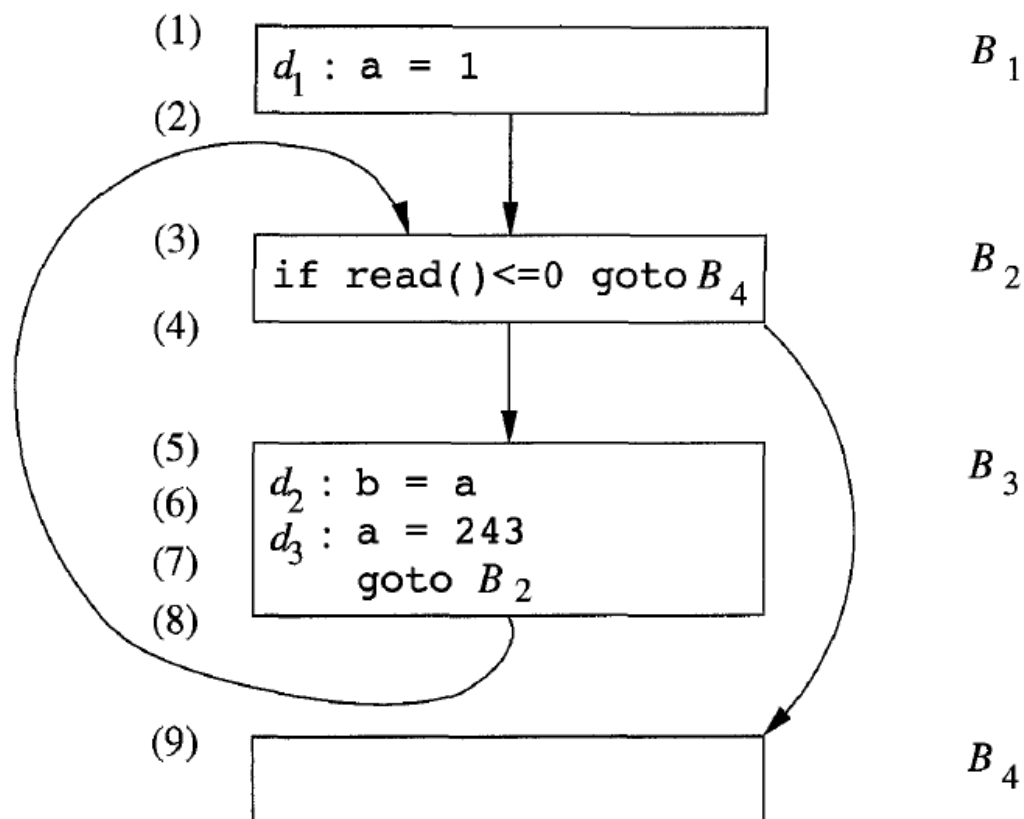
Problémy toku dát

- V čase kompilácie uvažujeme nad hodnotami premenných a výrazov počas behu.
- Vo všetkých zaujímavých bodoch chceme vedieť
 - Z ktorého príkazu priradenia pochádza hodnota premennej v tomto bode?
 - Ktoré premenné obsahujú hodnoty, ktoré nemajú po tomto bode použitie? (mŕtve premenné)
 - Aké sú možné hodnoty (range) premenných v tomto bode?

Cesty vykonávania programu

Cesta vykonávania programu – postupnosť bodov $p_1, \dots, p_n$, kde

- p_i je bod bezprostredne pred príkazom s a p_j je bod bezprostredne za príkazom s , alebo
- p_i je koniec bloku a p_j je začiatok niektorého z nasledujúcich blokov



Vo všeobecnosti
existuje nekonečne veľa
ciest vykonania
programu

Najkratšia: (1,2,3,4,9)

Druhá najkratšia:
(1,2,3,4,5,6,7,8,3,4,9)

Hodnoty toku riadenia a prenosové funkcie

Pre každý bod programu počítame **hodnoty (stavu) toku dát**

Závisia od konkrétnej úlohy

- Doména = množina všetkých možných hodnôt toku riadenia pre danú úlohu

Prenosová funkcia

- Nech D je doména, $f: D \rightarrow D$
- Pre príkaz: Vyjadruje sémantiku daného príkazu
 - $IN[s]$ – hodnoty pred vykonaním príkazu s
 - $OUT[s]$ – hodnoty po vykonaní príkazu s

Reaching definitions

V bode (5) dosahujúce definície pre premennú **a** sú **d₁** a **d₃**

Hodnoty:

- Množina definícií (priradení), ktoré dosiahnu daný bod programu

Doména

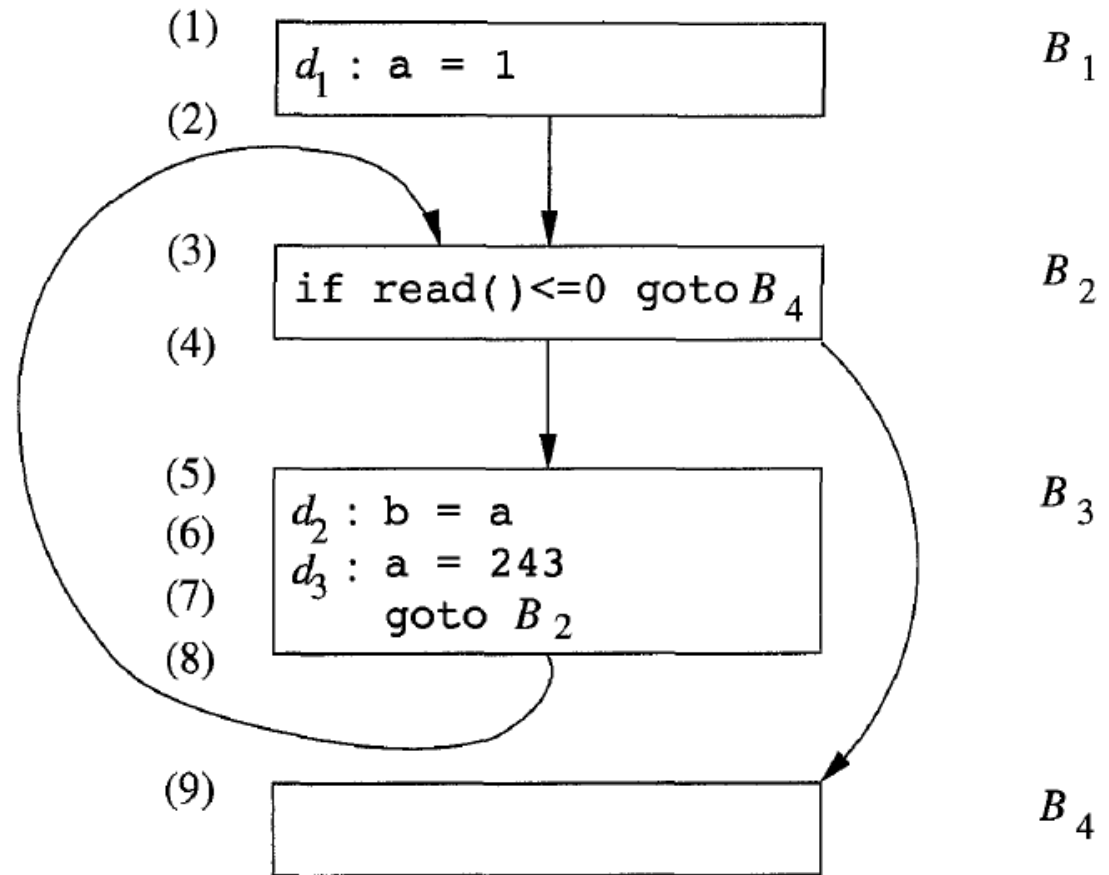
- Všetky podmnožiny množiny priradení programu

Príklad prenosovej funkcie

s: $x = y + z$

$OUT[s] = f_s(IN[s]) = \{s\} \cup (IN[s] \setminus \{\text{all previous definitions of } x\})$

Dopredná analýza – $OUT[s]$ sa počíta z $IN[s]$



Živost' premenných

Hodnoty:

- Množina premenných, ktoré sú živé v danom bode

Doména

- Všetky podmnožiny množiny premenných programu

Príklad prenosovej funkcie

$$s: x = y + z \quad \text{IN}[s] = f_s(\text{OUT}[s]) = (\text{OUT}(s) \setminus \{x\}) \cup \{y, z\}$$

Spätná analýza –

IN[s] sa počíta z OUT[s] (to isté sme robili v lokálnom algoritme pre živost')

Prenosová funkcia bloku

- Nech blok B pozostáva z príkazov $s_1, \dots, s_n$
- Pre príkazy v bloku platí $OUT[s_{i-1}] = IN[s_i]$

$IN[B]$ = hodnoty pre vstup bloku B

$OUT[B]$ = hodnoty pre výstup bloku B

$IN[B] = IN[s_1]$

$OUT[B] = OUT[s_n]$

Prenosová funkcia pre blok: $f_B = f_{s_n} \circ \dots \circ f_{s_2} \circ f_{s_1}$

Sémantika bloku a prechodu medzi blokmi (závisí od typu úlohy):

- **Dosahujúce definície:**

$OUT[B] = f_B(IN[B])$

$$IN[B] = \bigcup_{P \text{ a predecessor of } B} OUT[P].$$

- **Živosť premenných:**

$IN[B] = f_B(OUT[B])$

$$OUT[B] = \bigcup_{S \text{ a successor of } B} IN[S].$$

Definitely assigned variables

Počíta sa podmnožina premenných, ktoré majú v danom bode programu určite priradenú hodnotu

$OUT[B] = f_B(IN[B])$
(dopredná analýza)

$$IN(B) = \bigcap_{P \in preds(B)} OUT(P)$$

⇒ „**must**“ **analysis** (analýza nutného dosahu)

V predchádzajúcich úlohách malo sémanticky zmysel zjednotenie („**may**“ **analysis** – analýza možného dosahu)

Podobne funguje aj úloha „available expressions“.

Platnosť priradení

Reaching definitions vs available expressions

- Definícia premennej x je priradovací príkaz $x := \dots$.
- Priradenie môže byť zrejmé (unambiguous)
 - Priradovací príkaz $x := \dots$.
 - Načítanie do premennej x : $\text{read}(x)$, $\text{get}(x)$, $\dots$.
- Skryté (ambiguous) priradenia:
 - Priradenie cez smerník $*y := \dots$ (alebo priradenie prvku pola).
 - Volanie procedúry, ktorej argumenty sú volané referenciou.
 - Volanie procedúry používajúcej nelokálne premenné (side effect).
- Definícia d premennej x z bodu p programu môže platiť (reaches) v bode n programu. Ak existuje cesta z p do n , po ktorej sa nevyskytuje žiadne **zrejmé** priradenie premennej x .

Reaching definitions.

(„may“ analýza)

- Definícia d premennej x z bodu p programu musí platiť (is available) v bode n programu. Ak na žiadnej ceste z p do n , sa nevyskytuje žiadne **(ani skryté)** priradenie premennej x .

Available expressions.

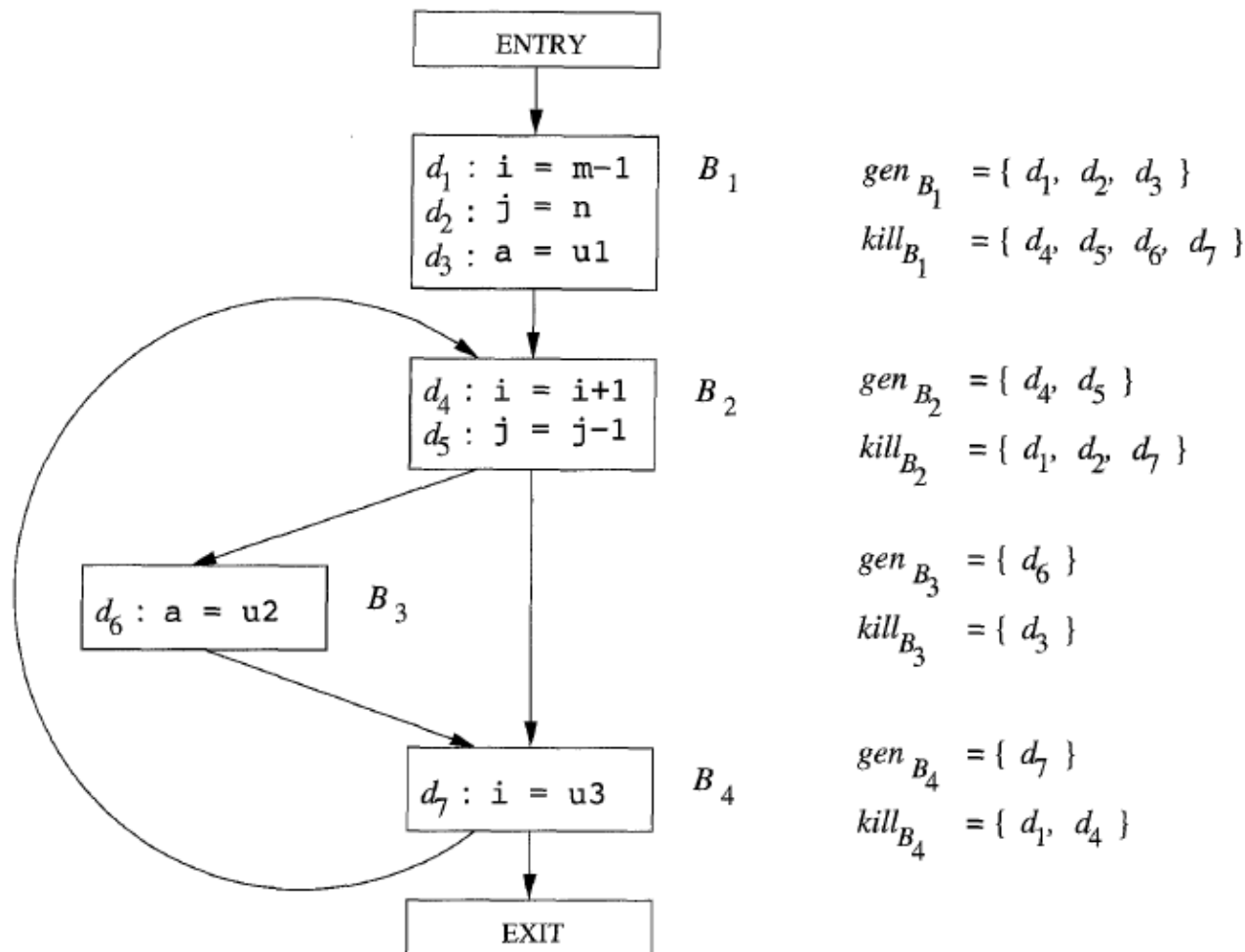
(„must“ analýza)

Príklad – reaching definitions

Pre príkaz: $OUT[s] = f_s(IN[s]) = gen_s \cup (IN[s] \setminus kill_s)$

Pre blok: $OUT[B] = f_B(IN[B]) = gen_B \cup (IN[B] \setminus kill_B)$

$OUT[ENTRY] = IN[B_1] = \emptyset$



Všetky ostatné definície i, j, a (kill sa dá vypočítať lokálne)

Iteratívny algoritmus

```
1)  OUT[ENTRY] =  $\emptyset$ ;  
2)  for (each basic block  $B$  other than ENTRY) OUT[ $B$ ] =  $\emptyset$ ;  
3)  while (changes to any OUT occur)  
4)      for (each basic block  $B$  other than ENTRY) {  
5)          IN[ $B$ ] =  $\bigcup_{P \text{ a predecessor of } B} \text{OUT}[P]$ ;  
6)          OUT[ $B$ ] =  $\text{gen}_B \cup (\text{IN}[B] - \text{kill}_B)$ ;  
      }
```

Iterujeme dovtedy, kým hodnoty IN a OUT nedokonvergujú

- Pre každé B , OUT[B] sa v ďalšej iterácii nikdy **nezmenšuje**
- Keďže doména je konečná, musí dôjsť k momentu, keď algoritmus dokonverguje

Live variables – postupovalo by sa podobne, ale inicializovalo by sa
IN[ENTRY] = $\emptyset$, IN[B] = $\emptyset$ a potom by sa počítali OUT[B] / IN[B] spätne.

Computation

Block B	$\text{OUT}[B]^0$	$\text{IN}[B]^1$	$\text{OUT}[B]^1$	$\text{IN}[B]^2$	$\text{OUT}[B]^2$
B_1	000 0000	000 0000	111 0000	000 0000	111 0000
B_2	000 0000	111 0000	001 1100	111 0111	001 1110
B_3	000 0000	001 1100	000 1110	001 1110	000 1110
B_4	000 0000	001 1110	001 0111	001 1110	001 0111
EXIT	000 0000	001 0111	001 0111	001 0111	001 0111

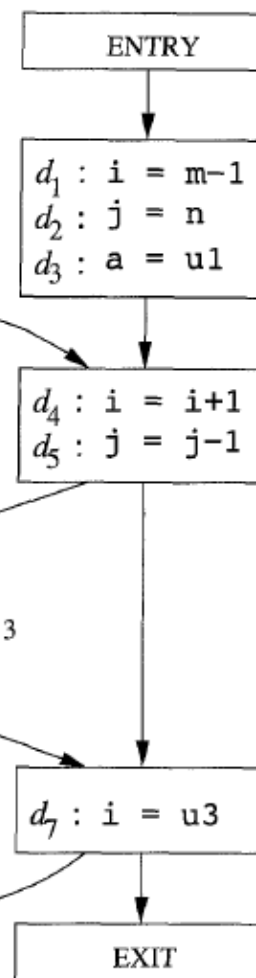
Množiny dosahujúcich definícií sú reprezentované ako 7-bitové vektory (bit i zľava reprezentuje definíciu d_i)

Potrebujeme 2 iterácie (+ inicializáciu)

- 1) $OUT[ENTRY] = \emptyset;$
- 2) **for** (each basic block B other than ENTRY) $OUT[B] = \emptyset;$
- 3) **while** (changes to any OUT occur)
- 4) **for** (each basic block B other than ENTRY) {
- 5) $IN[B] = \bigcup_{P \text{ a predecessor of } B} OUT[P];$
- 6) $OUT[B] = gen_B \cup (IN[B] - kill_B);$
- 7) }

Inicializácia

Iterácie



$gen_{B_1} = \{ d_1, d_2, d_3 \}$
 $kill_{B_1} = \{ d_4, d_5, d_6, d_7 \}$

$gen_{B_2} = \{ d_4, d_5 \}$
 $kill_{B_2} = \{ d_1, d_2, d_7 \}$

$gen_{B_3} = \{ d_6 \}$
 $kill_{B_3} = \{ d_3 \}$

$gen_{B_4} = \{ d_7 \}$
 $kill_{B_4} = \{ d_1, d_4 \}$

Block B	$OUT[B]^0$	$IN[B]^1$	$OUT[B]^1$	$IN[B]^2$	$OUT[B]^2$
B_1	000 0000	000 0000	111 0000	000 0000	111 0000
B_2	000 0000	111 0000	001 1100	111 0111	001 1110
B_3	000 0000	001 1100	000 1110	001 1110	000 1110
B_4	000 0000	001 1110	001 0111	001 1110	001 0111
EXIT	000 0000	001 0111	001 0111	001 0111	001 0111



Available expressions

```
OUT[ENTRY] =  $\emptyset$ ;  
for (each basic block  $B$  other than ENTRY)  $OUT[B] = U$ ;  
while (changes to any OUT occur)  
    for (each basic block  $B$  other than ENTRY) {  
         $IN[B] = \bigcap_{P \text{ a predecessor of } B} OUT[P]$ ;  
         $OUT[B] = e\_gen_B \cup (IN[B] - e\_kill_B)$ ;  
    }
```

Iterujeme dovtedy, kým hodnoty IN a OUT nedokonvergujú

- Pre každé B , $OUT[B]$ sa v ďalšej iterácii nikdy **nezväčšuje**
- Keďže doména je konečná, musí dôjsť k momentu, keď algoritmus dokonverguje

Formalizácia - úplné zväzy

Budeme reprezentovať doménu ako úplný zväz

	doména hodnôt	join (spojenie)	meet (priesek)	najväčší prvok	najmenší prvok	čiastočné usporiadanie	
$L =$	$<$	$L,$	$\sqcup,$	$\sqcap,$	$T,$	$\bot,$	$\sqsubseteq >$

Join: supremum, najmenšia horná závora

Meet: infimum, najväčšia dolná závora

$$x \sqcup x = x \text{ (idempotencia)}$$

$$x \sqcup y = y \sqcup x \text{ (komutatívnosť)}$$

$$x \sqcup (y \sqcup z) = (x \sqcup y) \sqcup z \text{ (asociatívnosť)}$$

$$x \sqsubseteq y \iff x \sqcup y = y$$

$$T \sqcup x = T, \bot \sqcup x = x$$

$$x \sqcap x = x \text{ (idempotencia)}$$

$$x \sqcap y = y \sqcap x \text{ (komutatívnosť)}$$

$$x \sqcap (y \sqcap z) = (x \sqcap y) \sqcap z \text{ (asociatívnosť)}$$

$$x \sqsubseteq y \iff x \sqcap y = x$$

$$T \sqcap x = x, \bot \sqcap x = \bot$$

Úplný zväz: Pre ľubovoľnú podmnožinu $A \subseteq L$ platí, že v L existuje $\sqcap A, \sqcup A$

Hasseov diagram

L = všetky podmnožiny $\{x,y,z\}$

Join: set union

Meet: set intersection

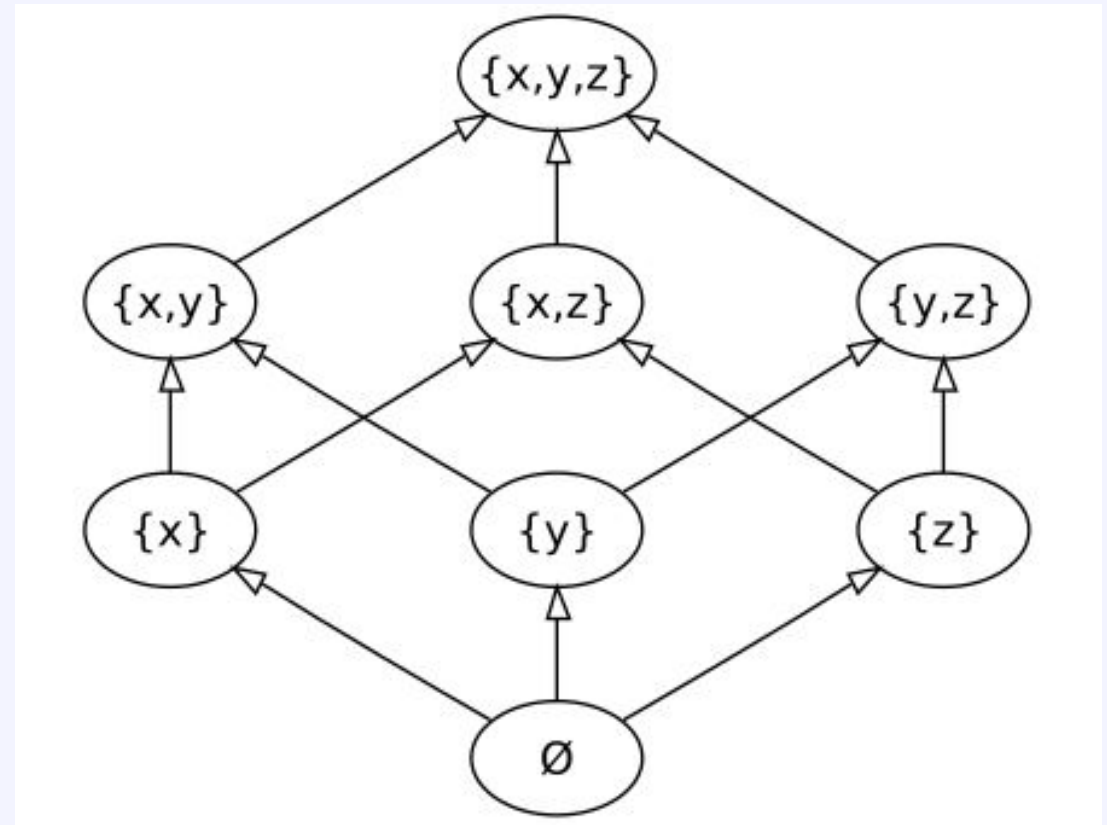
$a \sqsubseteq b$ if $a \cap b = a$

$a \sqsubseteq b$ if $a \cup b = b$

Order: $\subseteq$

$\perp = \emptyset$

$\top = \{x,y,z\}$



Hyperkocka

$L = \{ 0000, 0001, \dots, 1111 \}$ (štandardný booleovský zväz - hyperkocka)

Join: union (bitwise or)

Meet: intersection (bitwise and)

$x \sqsubseteq y$ if $(x \text{ bitwise and } y) = x$

$x \sqsupseteq y$ if $(x \text{ bitwise or } y) = y$

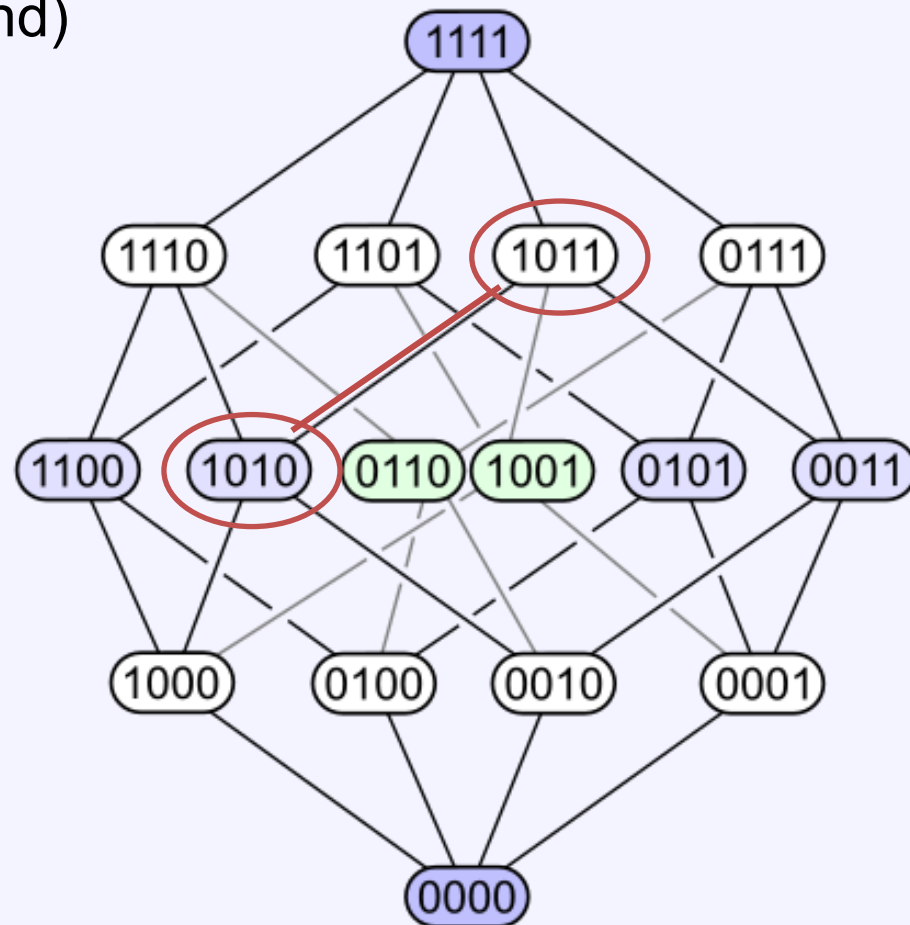
Order: bitwise subset order

$\perp = 0000$

$\top = 1111$

1010 bitwise and 1011 = 1010

1010 bitwise or 1011 = 1011



Hlavná myšlienka

- Zväz = množina všetkých možných abstraktných stavov programu
- Popísať transformácie pri vykonávaní programu zväzovými operáciami
 - Pre každý blok definujeme prenosovú funkciu $f_B: L \rightarrow L$.
 - Prechod medzi blokmi
- Ukázať, že tieto transformácie sú **monotónne**.
- Simuláciu behu programu
 - Doprednou (forward), alebo
 - Reverznou (backward)
- Určiť pevný bod
 - Monotónne transformácie garantujú, že v konečnom zväze dosiahneme pevný bod
 - Minimálne teoreticky su možné viaceré pevné body
- Pre väčšinu úloh je tento zväz priestor booleovských vektorov (hyperkocka) s operáciami a čiastočným usporiadaním po bitoch

Prenosové funkcie

- Prenosové funkcie charakterizujú účinok bloku B na informáciu o toku dát
- Trieda prenosových funkcií F
 - Musí obsahovať identickú funkciu (efekt prázdneho príkazu)
 - Byť uzavretá vzhľadom na kompozíciu (zložený príkaz)
 $\forall (f, g \in F)(h = f(g(x)) \in F)$
 - Všetky funkcie v F musia byť monotónne
 $\forall (f \in F)(f(x) \sqsubseteq x) \vee \forall (f \in F)(x \sqsubseteq f(x))$ $\begin{array}{l} \forall x, y: x \sqsubseteq y \Rightarrow f(x) \sqsubseteq f(y) \\ f(x \sqcup y) \sqsubseteq f(x) \sqcup f(y) \end{array}$
- Často bývajú tieto funkcie distributívne $f(x \sqcup y) = f(x) \sqcup f(y)$
- Distributívnosť implikuje monotónnosť
 $x \sqsubseteq y$ znamená $x \sqcup y = y$. Teda $f(x \sqcup y) = f(y)$. Podľa distributívnosti $f(x \sqcup y) = f(x) \sqcup f(y)$. Teda $f(x) \sqcup f(y) = f(y)$, to znamená $f(x) \sqsubseteq f(y)$.

Dopredná analýza toku dát

- Pre každý blok B je definované
 - $IN[B]$ informácia na vstupe do bloku b .
 - $OUT[B]$ informácia na výstupe z bloku b .
 - f_B prenosová funkcia bloku B .
- Riešenie musí spĺňať rovnice: (rovnice toku dát)

	$IN[B_0] = I_0$	precondition	} Monotónne transformácie
$\forall B$	$OUT[B] = f_B(IN[B])$		
$\forall B \neq B_0$	$IN[B] = \bigsqcup_{P \in \text{pred}(B)} OUT[P]$		

- Podľa (Tarskéhoho) vety existuje pevný bod:
Ak P je úplný zväz, tak každé monotónne zobrazenie $f: P \rightarrow P$ má pevný bod p , pre ktorý platí $f(p) = p$.
- Systém rovníc je možné iterovať podľa Kleeneho (naivná iterácia). Ukážeme aj efektívnejšiu metódu.

Reverzná analýza toku dát

- Pre každý blok B je definované
 - $IN[B]$ informácia na vstupe do bloku B .
 - $OUT[B]$ informácia na výstupe z bloku B .
 - rf_B reverzná prenosová funkcia bloku B
- Riešenie musí spĺňať rovnice: (rovnice toku dát)

$\forall B$	$IN[B] = rf_{B_n}(OUT[B])$	} Monotónne transformácie
$\forall B \neq B_f$	$OUT[B] = \bigsqcup_{P \in succ(B)} IN[P]$	
	$OUT[B_f] = O_f$ postcondition	

- Rovnice toku dát sa dajú zovšeobecniť aj pre množinu terminálnych bodov (viacero exitov).

Rovnice toku dát

- Úlohou kompilátoru je zostaviť rovnice toku dát pre jednotlivé úlohy analýzy toku dát
 - Reaching definitions forward
 - Available expressions forward
 - Live variables backward
 - Use-definition chain: ku každému použitiu premennej pridávame zoznam všetkých definícií, ktoré ju dosiahnu.
 - Definition-use chain: naopak ku každej definícii zoznam miest programu, kde je použitá.
- Iné atypické problémy
 - Sign analysis (znamienková analýza)
 - Aké znamienka môže premenná nadobúdať
- Riešenie rovníc je delegované na samostatný program, od ktorého si optimalizátor prevezme výsledky (SoC, modular design)
 - Presnejšie kladie mu dotazy

Metódy riešenia rovníc toku dát

Naivná iterácia – round robin (A. Kildall. 1971)

Initialize: for all B do { $OUT[B] := \perp$; $IN[B] = \perp$; }

$IN[B_0] = I_0$; /* alebo $OUT[B_f] = O_f$; */

Iterate: **repeat**

for all equations **do** compute equation;

until change occurred;

$\angle$ -polozväz (horný):

- Inicializácia $IN[n]$, $OUT[n]$ na najmenší prvok $\perp$
- Konverguje k najmenšiemu pevnému bodu (vzhľadom na reláciu usporiadania)

Worklist algoritmus pre dopredné DFE

Naivná iterácia opakuje mnohé výpočty zbytočne. Efektívnejšiu implementáciu dosiahneme, keď si počas práce algoritmu budeme udržiavať pracovný zoznam (*worklist*) uzlov, ktorých sa zmeny týkajú.

```
for each B do OUT[B] :=  $f_B(\perp)$ ;  
IN[B0] := I0; OUT[B0] :=  $f_{B_0}(I_0)$ ;  
worklist := B - { B0 };  
while worklist ≠ ∅ do  
    { remove a block B from worklist;  
      IN[B] =  $\bigsqcup_{P \in \text{pred}(B)} \text{OUT}[P]$ ;  
      OUT[B] =  $f_B(\text{IN}[B])$ ;  
      if OUT[B] changed then worklist := worklist ∪ succ(B);  
      /* all successors of n must be added to the worklist */  
    }
```

- V programe je zamlčaná implementácia worklist. Používa sa zásobník, fronta (queue) a prioritná fronta (musí sa použiť implementácia množiny).

Worklist algoritmus pre reverzné DFE

```
for each B do IN[B] := rfB(⊥);  
for each B ∈ Bfinal do { OUT[B] := Of; IN[B] := rfB(Of); }  
worklist := B - Bfinal;  
while worklist ≠ ∅ do  
    { remove a block B from worklist;  
      OUT[B] =  $\bigsqcup_{P \in \text{succ}(B)} \text{IN}[P]$ ;  
      IN[B] = rfB(OUT[B]);  
      if IN[B] changed then worklist := worklist ∪ pred(B);  
        /* all predecessors of B must be added to the worklist */  
    }
```

- V tomto prípade program zohľadňuje možnosť viacerých terminálnych uzlov.

Správnosť algoritmu

- Konštrukcia algoritmu zabezpečuje, že:
 - Pre každý uzol platí vzťah medzi vstupom a výstupom (dopredne alebo spätne).
 - Ak sa pri výpočte uzlu zmenia hodnoty, tak sa jeho následníci (predchodcovia pri reverznom spracovaní) dostanú znova do zoznamu spracovania (worklist). Po ich vybraní zo zoznamu sa prepočítajú ich hodnoty.
- Znamená to, že program môže skončiť, iba ak sú splnené rovnice toku dát.
- Skončenie:
 - Každá rovnica pre uzol je monotónna. To zaručuje, že niekedy nastane prípad, že hodnota v uzle sa už pri ďalších výpočtoch nebude meniť. Podľa Tarského vety.
 - Každý uzol sa raz stabilizuje. Keď sa stabilizujú všetky uzly, výpočet skončí.

Úlohy analýzy toku dát

- Na špecifikáciu úlohy analýzy toku riadenia treba určiť:
 1. Doménu (obor definície)
 2. Optimalizačný polozväz
 3. Prenosové funkcie
 4. Inicializáciu
- Množina rovníc je už určená grafom toku riadenia a predošlými špecifikáciami.

Dátové štruktúry pre výpočet

- Potrebujeme reprezentovať množiny definícií (resp. premenných)
 - Vhodnou reprezentáciou je powerset (bitový vektor)
 - Operácie $\cap$ a $\cup$ sú boolovské operácie po bitoch.} Vid' príklad hyperkocky
- Doména L je množina všetkých podmnožín množiny všetkých definícií / premenných v programe.
 - $\sqcup$ je zjednotenie $\cup$
 - $\sqcap$ je prienik $\cap$
 - $\sqsubseteq$ je množinová inklúzia $\subseteq$
 - $\perp$ je prázdna množina $\emptyset$
 - $\top$ je množina všetkých definícií D
- Uvedená štruktúra je booleovský (úplný) zväz.

Pre jednosmerné úlohy stačí horný (resp. dolný) polozväz = optimalizačné polozväzy. Jednotné vyjadrenie analýz:

$$\text{OUT}[b] = f_b \left(\bigsqcup_{p \in \text{pred}(b)} \text{OUT}[p] \right), \text{ reverzné opačne}$$

Formalizácia "reaching definitions"

„May“ dopredná analýza

- L = potenčná množina všetkých definícií v programe (množina všetkých podmnožín množiny všetkých definícií v programe)
- $\sqcup = \cup$ (relácia usporiadania je $\subseteq$)
$$x \sqsubseteq y \Leftrightarrow x \sqcup y = y$$
$$D_1 \subseteq D_2 \Leftrightarrow D_1 \cup D_2 = D_2$$
- $\perp = \emptyset$
- $I_0 = \text{IN}[B_0] = \emptyset$, $\text{OUT}[B] = \perp$
- F = množina všetkých funkcií tvaru $f_B(x) = \text{gen}_B \cup (x \setminus \text{kill}_B)$
 - *kill* je množina všetkých definícií, ktoré sú v bloku zrušené
 - *gen* je množina definícií, ktoré blok generuje.

Formalizácia "available expressions"

„Must“ dopredná analýza

- L = potenčná množina všetkých definícií v programe (množina všetkých podmnožín množiny všetkých definícií v programe)

$$\begin{aligned}x \sqsubseteq y &\Leftrightarrow x \sqcap y = x \\D_1 \supseteq D_2 &\Leftrightarrow D_1 \cup D_2 = D_1\end{aligned}$$

- $\sqcap = \cap$ (relácia usporiadania je $\supseteq$)
- $\perp$ = množina všetkých definícií v programe
- $I_0 = \text{IN}[B_0] = \emptyset$, $\text{OUT}[B] = \perp$
- F = množina všetkých funkcií tvaru $f_B(x) = \text{gen}_B \cup (x \setminus \text{kill}_B)$
 - *kill* je množina všetkých definícií, ktoré sú v bloku zrušené
 - *gen* je množina definícií, ktoré blok generuje.

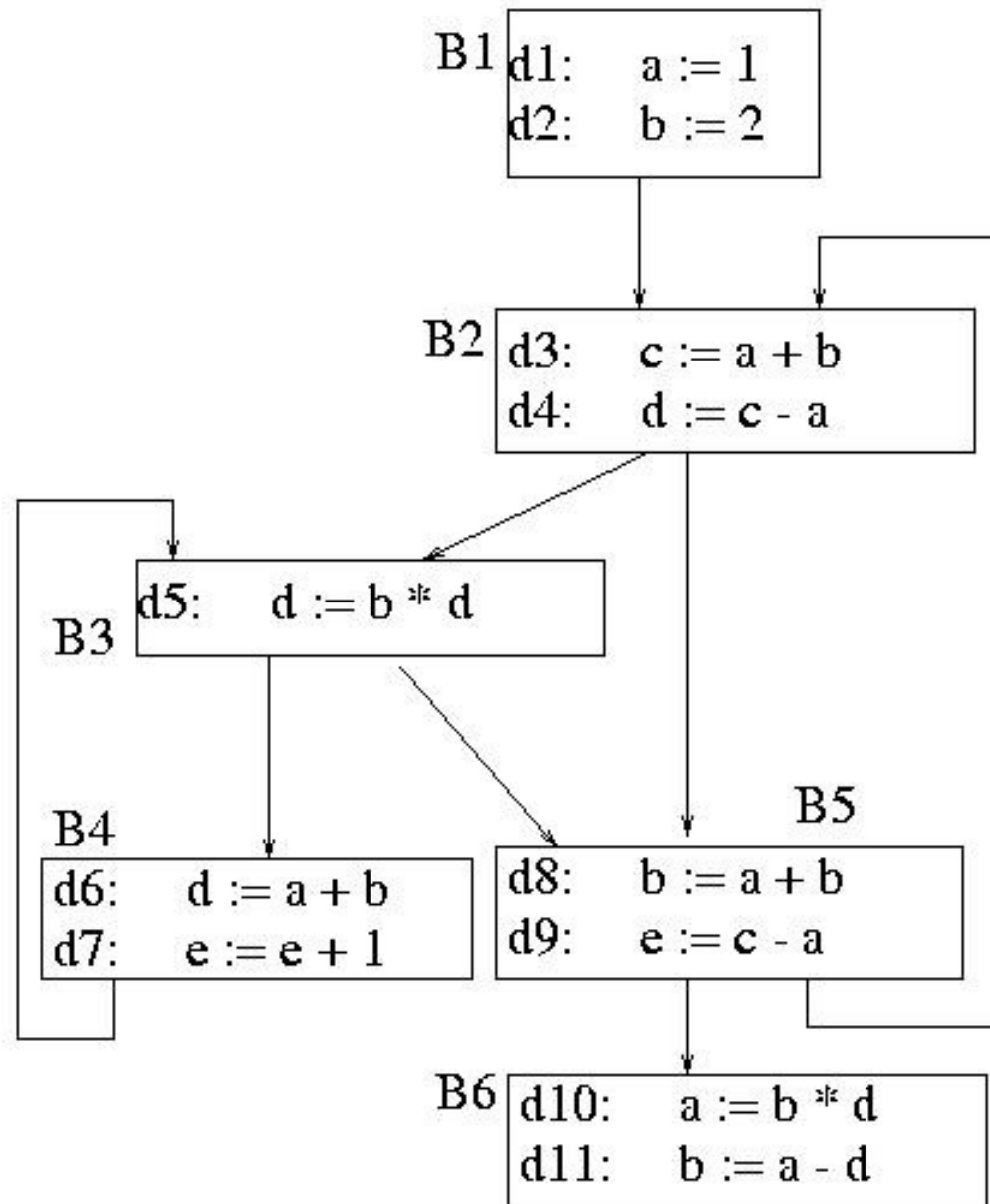
Formalizácia "live variables"

„May“ reverzná analýza

- L = potenčná množina všetkých premenných v programe (množina všetkých podmnožín množiny všetkých premenných v programe)
- $\sqcup = \cup$ (relácia usporiadania je $\subseteq$)
- $\perp = \emptyset$
- $OUT[B_f] = O_f = \emptyset, IN[B] = \perp$
- F = množina všetkých funkcií tvaru $rf_B(x) = use_B \cup (x \setminus def_B)$
 - def je množina premenných, ktorým blok priraduje hodnotu skôr ako ich použije
 - use je množina premenných, ktoré blok používa skôr ako ich definuje

Príklad

Block	DEF	USE
B1	{a,b}	{ }
B2	{c,d}	{a,b}
B3	{ }	{b,d}
B4	{d}	{a,b,e}
B5	{e}	{a,b,c}
B6	{a}	{b,d}



DFA úlohy z dračej knihy

	Reaching Definitions	Live Variables	Available Expressions
Domain	Sets of definitions	Sets of variables	Sets of expressions
Direction	Forwards	Backwards	Forwards
Transfer function	$gen_B \cup (x - kill_B)$	$use_B \cup (x - def_B)$	$e_gen_B \cup (x - e_kill_B)$
Boundary	$OUT[ENTRY] = \emptyset$	$IN[EXIT] = \emptyset$	$OUT[ENTRY] = \emptyset$
Meet ($\wedge$)	$\cup$	$\cup$	$\cap$
Equations	$OUT[B] = f_B(IN[B])$ $IN[B] = \bigwedge_{P, pred(B)} OUT[P]$	$IN[B] = f_B(OUT[B])$ $OUT[B] = \bigwedge_{S, succ(B)} IN[S]$	$OUT[B] = f_B(IN[B])$ $IN[B] = \bigwedge_{P, pred(B)} OUT[P]$
Initialize	$OUT[B] = \emptyset$	$IN[B] = \emptyset$	$OUT[B] = U$

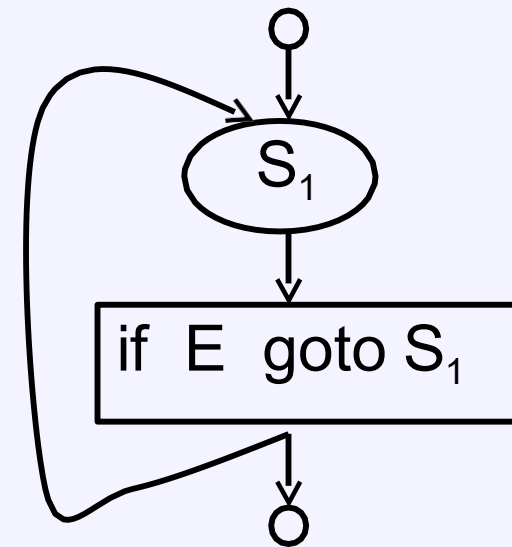
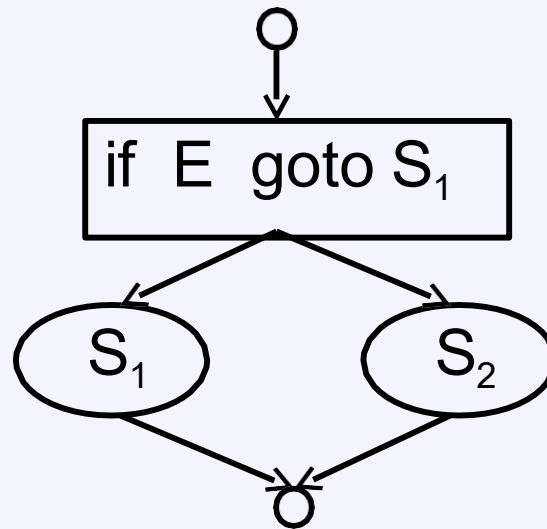
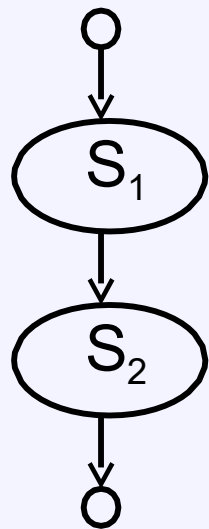
Pozn: Ullman všetky úlohy formuluje pre dolný polozväz a hľadá maximálny pevný bod.

Duálny problém horný polozväz a minimálny pevný bod. Úplný zväz potrebujeme len pre obojsmerné problémy.

K notácii: $OUT[ENTRY] = IN[B_0]$, $IN[EXIT] = OUT[B_f]$

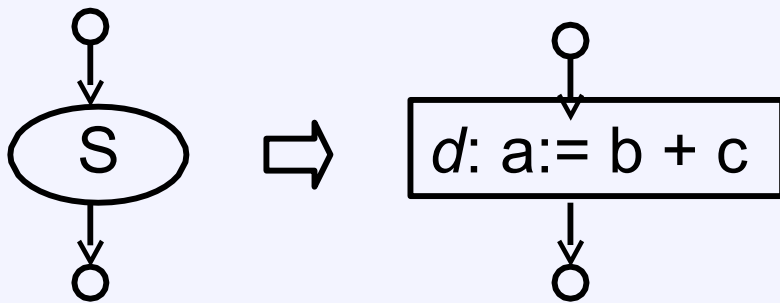
Syntaxou riadený preklad (reaching definitions)

$S \rightarrow$ **id** **':='** E
| S_1 **';** S_2
| **if** E **then** S_1 **else** S_2
| **do** S **while** E



Rovnice toku dát

- V každom príkaze (bloku S):
 - Niečo vznikne (je nejaké priradenie) – $\text{gen}[S]$
 - Eventuálne nejaké priradenie prestane platiť – $\text{kill}[S]$
- Každý príkaz (blok) má atribúty:
 - Zdedený atribút $\text{in}[S]$
 - Syntetizované atribúty $\text{out}[S]$, $\text{gen}[S]$ a $\text{kill}[S]$
- Platí rovnica (prenosová funkcia) príkazu (bloku):
$$\text{out}[S] = \text{gen}[S] \cup (\text{in}[S] \setminus \text{kill}[S])$$
 - $\text{gen}[S]$ obsahuje len posledné priradenia premennej bloku.
 - $\text{kill}[S]$ sa vzťahuje len na definície do bloku vchádzajúce.



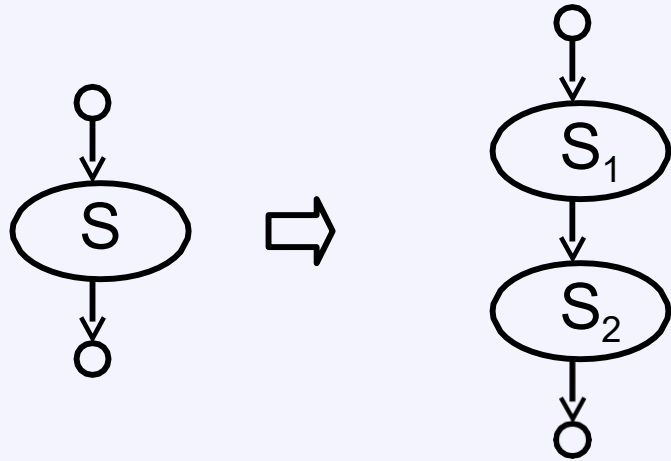
$$\text{gen}[S] = \{d\}$$

$$\text{kill}[S] = D_a - \{d\}$$

$$\text{out}[S] = \text{gen}[S] \cup (\text{in}[S] - \text{kill}[S])$$

Kde D_a je množina všetkých doterajších definícií premennej a .

Zložený a podmienený príkaz



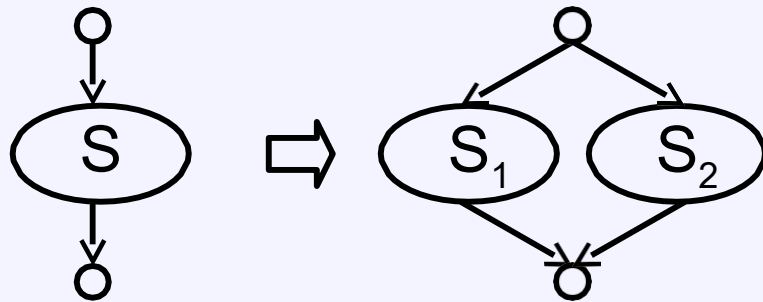
$$\text{gen}[S] = \text{gen}[S_2] \cup (\text{gen}[S_1] - \text{kill}[S_2])$$

$$\text{kill}[S] = \text{kill}[S_2] \cup (\text{kill}[S_1] - \text{gen}[S_2])$$

$$\text{in}[S_1] = \text{in}[S]$$

$$\text{in}[S_2] = \text{out}[S_1]$$

$$\text{out}[S] = \text{out}[S_2]$$



$$\text{gen}[S] = \text{gen}[S_1] \cup \text{gen}[S_2]$$

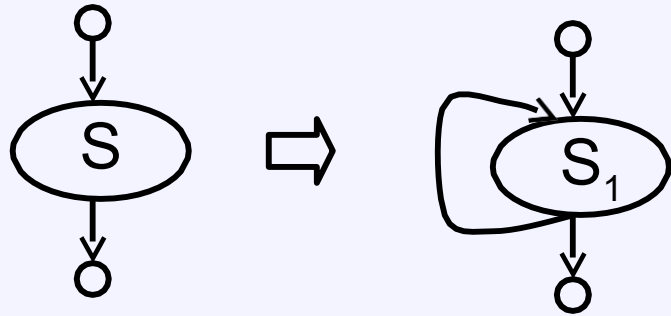
$$\text{kill}[S] = \text{kill}[S_1] \cap \text{kill}[S_2]$$

$$\text{in}[S_1] = \text{in}[S]$$

$$\text{in}[S_2] = \text{in}[S]$$

$$\text{out}[S] = \text{out}[S_1] \cup \text{out}[S_2]$$

Cyklus a jeho stabilizácia



$$\text{gen}[S] = \text{gen}[S_1]$$

$$\text{kill}[S] = \text{kill}[S_1]$$

$$\text{in}[S_1] = \text{in}[S] \cup \text{gen}[S_1]$$

$$\text{out}[S] = \text{out}[S_1]$$

$$\text{in}[S_1] = \text{in}[S] \cup \text{out}[S_1]$$

$$\text{out}[S_1] = \text{gen}[S_1] \cup (\text{in}[S_1] - \text{kill}[S_1])$$

$$i = j \cup o$$

$$o = g \cup (i - k)$$

Predpokladáme $o_0 = \emptyset$

$$i_1 = j \cup o_0 = j$$

$$o_1 = g \cup (i_1 - k) = g \cup (j - k)$$

$$i_2 = j \cup o_1 = j \cup g \cup (j - k) = j \cup g$$

$$o_2 = g \cup (i_2 - k) = g \cup (j \cup g - k) = g \cup (j - k)$$